

Computer Vision
Assignment 06

Semantic Segmentation

Melchor Lafuente
Iker Jauregui

Contents

1	Introduction	2
2	Exploratory Data Analysis (EDA)	2
3	Experimental Setup	3
3.1	Object Clusterization	3
3.2	Callbacks and LR Scheduler	4
3.3	Model Architectures	4
3.4	Evaluation Metrics	4
4	Experiments	5
4.1	Experiment 1: Unet	5
4.2	Experiment 2: DeepLabV3	6
4.3	Experiment 3: SegFormer	7
4.4	Experiment 4: TTA	9
5	Conclusions	10
6	Future Work	10

1 Introduction

The aim of this project is to benchmark different semantic segmentation architectures over the *Clothing Co-Parsing (CPP)* dataset.

2 Exploratory Data Analysis (EDA)

An Exploratory Data Analysis was performed to understand the dataset characteristics and guide experiment development. The following key observations were made:

- The provided dataset is compounded by 1007 samples which are divided in three different splits: *train* (701 samples), *valid* (151 samples) and *test* (155 samples).
- As we can see in Figure 1, there is a total of 59 classes and the dataset is very imbalanced.

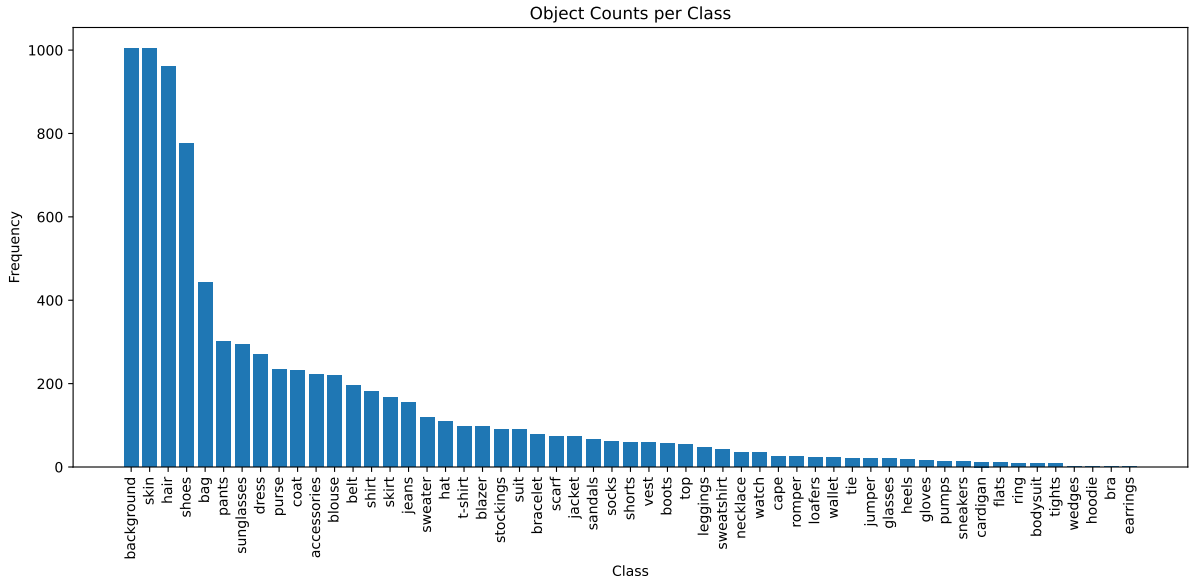


Figure 1: Object class distribution in the original dataset.

3 Experimental Setup

3.1 Object Clusterization

We considered that the original dataset had too many different classes with very few samples, so we decided to group the original classes into 6 different clusters. As can be seen at Table 1, the clusterization was made by grouping similar types of clothes or objects. The resulting dataset is much more balanced (Figure 2).

Label	Cluster	Classes
0	no_clothing	background, skin, hair
1	accessories	belt, scarf, hat, gloves, bag, purse, wallet, accessories, bracelet, necklace, watch, ring, earrings, tie, sunglasses, glasses, intimate
2	upper_wear	shirt, t-shirt, blouse, top, sweater, hoodie, sweatshirt, jacket, coat, blazer, cardigan, vest, cape, jumper, bra
3	one_piece_wear	dress, romper, bodysuit, suit, swimwear
4	footwear	shoes, sandals, boots, sneakers, flats, heels, pumps, wedges, loafers, socks, clogs
5	lower_wear	pants, jeans, skirt, shorts, leggings, tights, stockings, panties

Table 1: Clusters’ composition.

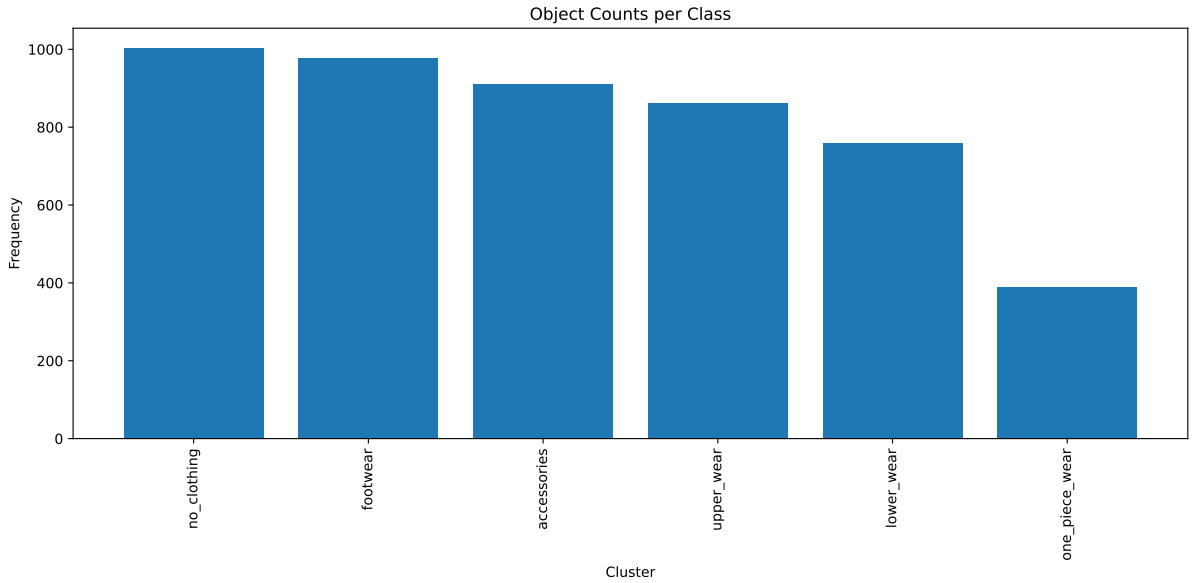


Figure 2: Clusterized object distribution sorted by frequency.

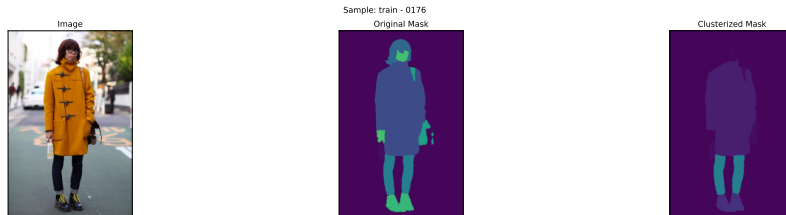


Figure 3: Original masking (middle) vs new clusterized masking (right).

3.2 Callbacks and LR Scheduler

During the experiments, two main callbacks were used: **EarlyStopping** and **ModelCheckpoint**. Instead of using a `LearningRateFinder`, we decided to employ a learning rate scheduler. We decided to use the **ReduceLROnPlateau** scheduler, as it could lead the models to obtain the best possible validation losses. Finally, we fixed the batch size to 32 not to overload the shared GPU.

3.3 Model Architectures

In this project we decided to test three architectures: *Unet*, *DeepLabV3* and *SegFormer*. This selection was made based on the differences between them.

Unet is a classic CNN-based encoder-decoder architecture with skip connections to preserve local details. In contrast, *DeepLabV3* employs atrous convolutions and Atrous Spatial Pyramid Pooling (ASPP) for capturing multi-scale global context without a symmetric decoder. *SegFormer*, however, shifts to a transformer paradigm, using hierarchical self-attention mechanisms for efficient, long-range dependency modeling. These differences span from traditional convolutional approaches to modern attention-based designs, allowing us to evaluate diverse segmentation strategies.

3.4 Evaluation Metrics

For the evaluation metrics, we used *pixel accuracy*, *intersection over union* and *Dice coefficient* (F1 score).

4 Experiments

4.1 Experiment 1: Unet

As we can see in the loss curves (Figure 4), the training run quite smoothly, keeping a consistent gap between train and validation. The model actually learned and didn't overfit.

If we look to the semantic segmentation generated (Figure 5), we can see that the model's performance is quite good.

However, if we analyze both global (Table 2) and per class (Table 3) metrics, we find that the results are far from optimal. This behavior seems to be directly related to each classes' pixel density, because *no_clothing* class (composed by background, hair and skin) shows the best performance while *one_piece_wear* (least frequent class) and *accessories* (class with smallest objects) show the worst. This means that, even if we tried to balance the dataset by clusterizing the original object classes, we didn't balance it enough respect to the pixel amount or density.

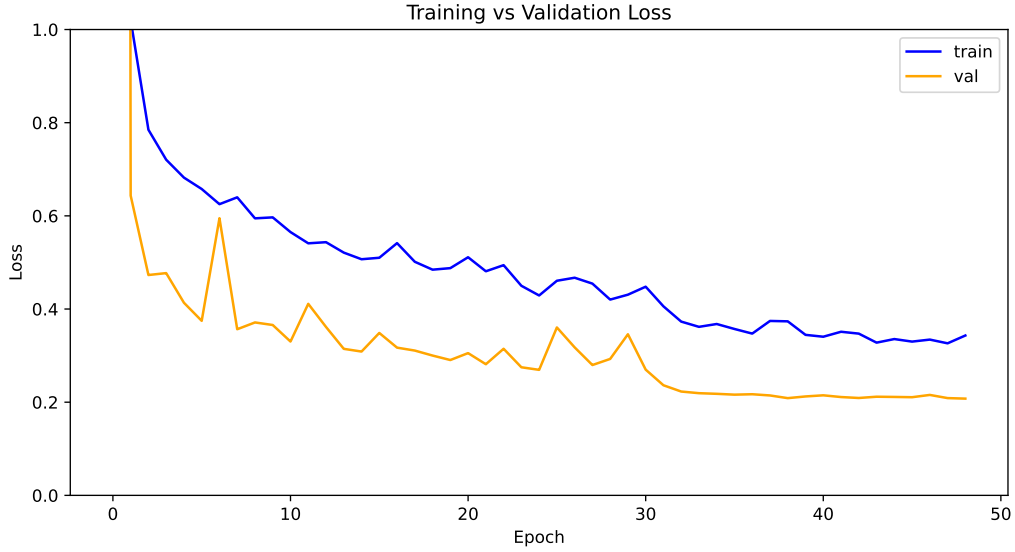


Figure 4: Train and validation losses for the Unet.

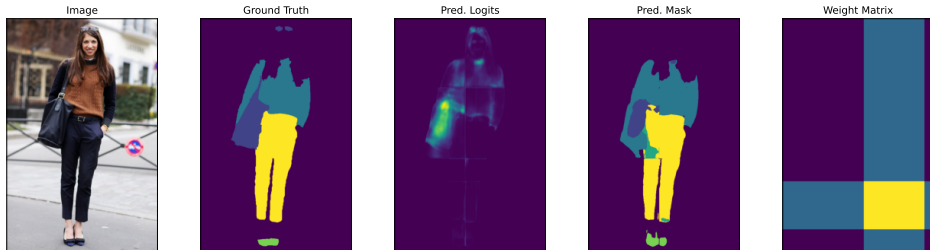


Figure 5: Semantic segmentation generated by the Unet on a test sample.

Pixel accuracy	IoU (macro)	Dice (macro)
0.912795	0.482819	0.555344

Table 2: Global metrics of Unet on the Test set.

Label	Cluster	IoU	Dice	Stdev IoU	Stdev Dice
0	no_clothing	0.964026	0.981176	0.041963	0.023946
1	accessories	0.230452	0.320262	0.237895	0.286325
2	upper_wear	0.552506	0.646452	0.314381	0.329074
3	one_piece_wear	0.120234	0.164572	0.212539	0.268294
4	footwear	0.609142	0.730069	0.217529	0.204537
5	lower_wear	0.411789	0.480077	0.367614	0.403097

Table 3: Metrics per class of Unet on Test set.

4.2 Experiment 2: DeepLabV3

The results obtained in this experiment are very similar to the previous one (Figure 6 and Figure 7). There is only a slight difference respect to the global (Table 4) and per class (Table 5) metrics, but not significant enough to find a reason to justify it.

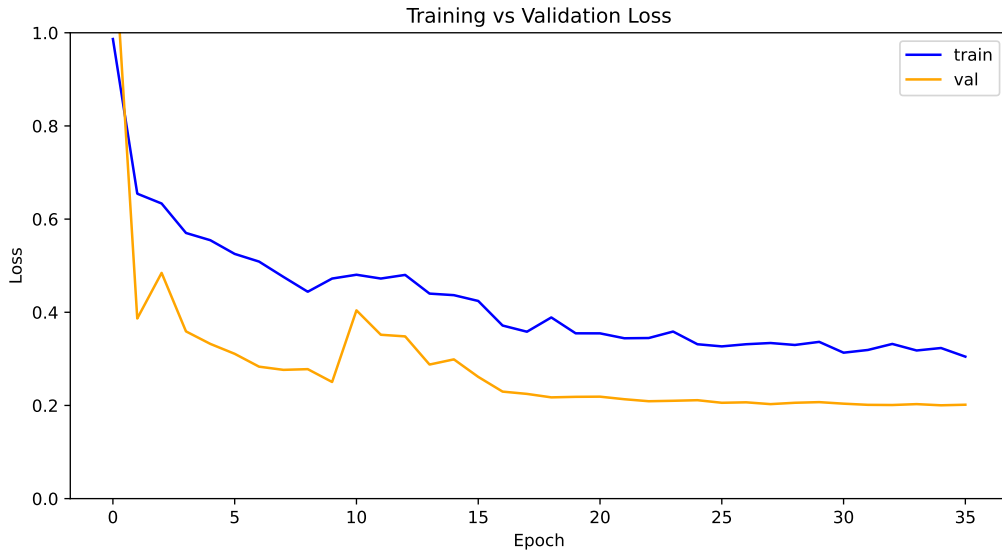


Figure 6: Train and validation losses for the DeepLabV3.

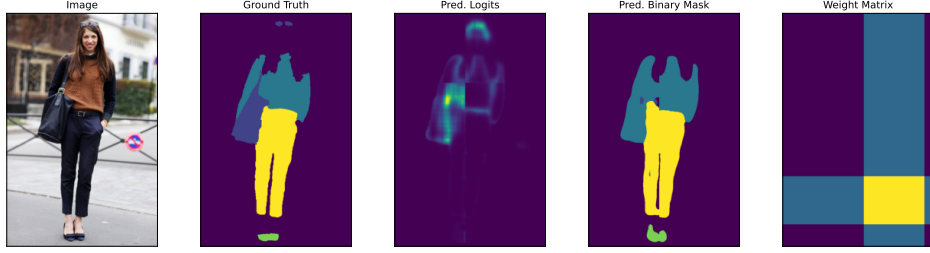


Figure 7: Semantic segmentation generated by the DeepLabV3 on a test sample.

Pixel accuracy	IoU (macro)	Dice (macro)
0.915002	0.484669	0.558722

Table 4: Global metrics of DeepLabV3 on the Test set.

Label	Cluster	IoU	Dice	Stdev IoU	Stdev Dice
0	no_clothing	0.963348	0.981100	0.029008	0.015791
1	accessories	0.205223	0.283340	0.239811	0.292976
2	upper_wear	0.539989	0.636478	0.310977	0.327530
3	one_piece_wear	0.137900	0.184833	0.229484	0.289357
4	footwear	0.579210	0.702491	0.225605	0.222593
5	lower_wear	0.395302	0.467320	0.357488	0.394218

Table 5: Metrics per class of DeepLabV3 on Test set.

4.3 Experiment 3: SegFormer

Once again, comparing to the previous experiments, we don't see any big differences on the predicted masks (Figure 5). However, this time we can see that the obtained metrics (Table 6 and Table 7) are worse than the obtained with the previous models. The reason behind this could be that this time the model's training stopped too early and didn't learn as good as before (Figure 8).

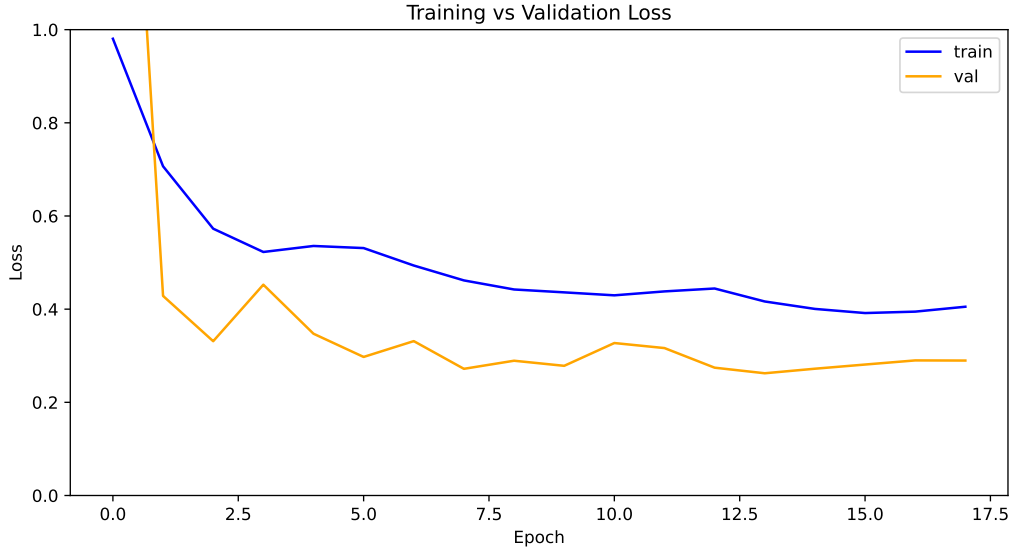


Figure 8: Train and validation losses for the SegFormer.

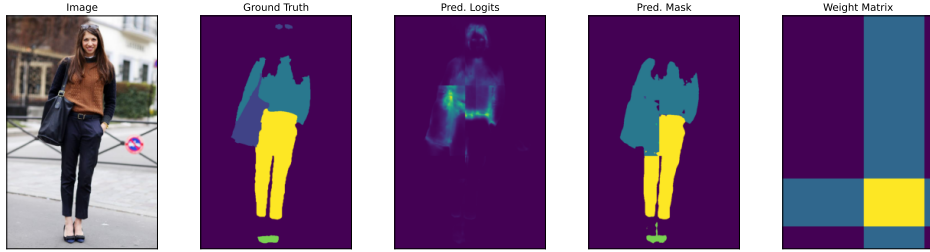


Figure 9: Semantic segmentation generated by the SegFormer on a test sample.

Pixel accuracy	IoU (macro)	Dice (macro)
0.901348	0.439118	0.513290

Table 6: Global metrics of SegFormer on the Test set.

Label	Cluster	IoU	Dice	Stdev IoU	Stdev Dice
0	no_clothing	0.948233	0.972802	0.046114	0.026713
1	accessories	0.141418	0.198385	0.212653	0.269725
2	upper_wear	0.486888	0.596377	0.285352	0.311494
3	one_piece_wear	0.124364	0.177076	0.196523	0.257164
4	footwear	0.541002	0.662154	0.250411	0.251537
5	lower_wear	0.355433	0.431262	0.337354	0.380958

Table 7: Metrics per class of SegFormer on Test set.

4.4 Experiment 4: TTA

Taking into account the three trained models' performance, we can see that the best results were obtained by Unet and DeepLabV3. If we look closely to the metrics per class (Table 3 and Table 5), we can see that Unet is slightly better, so for the next TTA experiment we selected this architecture.

Various transformation configurations were tested and, in most of the cases, the obtained results were worse compared to not using TTA. Even if this can seem strange, this behavior could have the following explanations:

- *VerticalFlip*: Under this context, this transformation doesn't have any sense at all, because the position of the clothes normally follow a specific order (hats on top, footwear at bottom, etc.). Flipping vertically the image could confuse the models.
- *Rotation90*: Same as previous transformation, applying big rotations to the image could locate clothes on unnatural places, leading models to confusion.
- *Add* and *Multiply*: Unlike the previous transformations, these are photometric transformations and only modify pixels' intensity instead of their position. So, at first thought, they should have improved model's performance, but they actually didn't. The reason beneath it may be that we didn't use any data augmentation techniques during training time, so the model may have related some colors to specific classes.

The only improvement we saw was with the *HorizontalFlip* transformation. That makes perfect sense with the explanations we recently gave, because flipping horizontally the image doesn't perturb the natural order of the clothes and doesn't modify pixel intensities. Here are the final metrics obtained with this transformation:

Method	Pixel accuracy	IoU (macro)	Dice (macro)
No TTA	0.912795	0.482819	0.555344
With TTA	0.914365	0.486993	0.559290

Table 8: Comparison between global metrics of the Unet with and without TTA.

Label	Cluster	IoU (No TTA)	IoU (With TTA)	Dice (No TTA)	Dice (With TTA)
0	no_clothing	0.964026	0.964844	0.981176	0.981633
1	accessories	0.230452	0.240038	0.320262	0.332759
2	upper_wear	0.552506	0.555317	0.646452	0.648990
3	one_piece_wear	0.120234	0.126183	0.164572	0.170652
4	footwear	0.609142	0.608691	0.730069	0.725661
5	lower_wear	0.411789	0.413557	0.480077	0.481740

Table 9: Comparison of per class metrics of the Unet with and without TTA.

As we can see in the previous tables (Table 8 and Table 9), the metrics were improved for almost all the classes (all except *footwear*).

5 Conclusions

The experiments conducted in this project revealed that, despite the clusterization of the original 59 classes into 6 broader categories, the models didn't achieve really good metrics. This behavior is directly related to the class pixel density imbalance present in the dataset, where *no_clothing* dominates the pixel distribution while classes like *one_piece_wear* and *accessories* represent much smaller portions. This imbalance prevented the models from learning robust representations for minority classes.

Additionally, the TTA experiments demonstrated that the use of certain transformations can be prejudicial when they generate samples that didn't be seen during training time. Transformations like *VerticalFlip*, *Rotation90*, and photometric variations (*Add*, *Multiply*) actually degraded performance, as they introduced unrealistic configurations (e.g., flipped persons, unnatural color variations) that the models were not prepared to handle.

6 Future Work

Based on the findings of this project, the following improvements are proposed:

- **Balance dataset according to class pixel densities:** Implement techniques such as weighted sampling or synthetic minority oversampling to address the severe pixel-level class imbalance.
- **Retrain models with data augmentation:** Incorporate data augmentation techniques during training time, including horizontal flips, color jittering, and intensity variations, so that models become robust to the transformations used in TTA.